SuperCollider Mastery: Exercise 01

Building Self-Cleaning Instruments & Pitch-Mapped Sequences

> **Objective:** Bridge your structural outline in l01_idiomaticSC.txt with the complete synthesizer and pattern setup in fur_elise.txt. Focus on self-releasing synths, MIDI note conversion, sequential timing, and stereo harmonic layering.

EXERCISE 01 TASKS

[ ] Task 1: Self-Cleaning SynthDef

Define a SynthDef (e.g. \simplePluck) with arguments for out, freq, amp, and release.
Wrap your SinOsc.ar inside a percussive envelope using EnvGen.kr and Env.perc.
Key Requirement: Set doneAction: 2 so the synth node despawns automatically from scsynth when the envelope finishes.

[ ] Task 2: MIDI Pitch Mapping

Store the 9-note opening motif of *Für Elise* in an array of MIDI pitch integers:
Pitches: E5 (76) → D#5 (75) → E5 (76) → D#5 (75) → E5 (76) → B4 (71) → D5 (74) → C5 (72) → A4 (69).
Convert MIDI numbers to cycles per second (Hz) dynamically using SuperCollider's .midicps method when passing values to \freq.

[ ] Task 3: Sequential Timing

Write a Routine that iterates through your MIDI note array using .do.
Instantiate your SynthDef for each note and call .wait (e.g. 0.25.wait) inside the loop to schedule notes in time rather than playing them all simultaneously.

[ ] Task 4: Stereo Spatialize & Harmonic Layering

Expand your synth definition to give it organic warmth and width:
1. Harmonics: Add a second subtle overtone generator (e.g. SinOsc.ar(freq * 2) * 0.25) to your main oscillator.
2. Stereo Panning: Pass the combined signal into Pan2.ar(sig, pan) so the sound spreads across stereo channels.

QUICK REFERENCE & CODE HINTS

Concept / Mechanism	SuperCollider Syntax	Role in fur_elise.txt
Automatic Cleanup	doneAction: 2	Frees synth node on envelope end

Concept / Mechanism	SuperCollider Syntax	Role in fur_elise.txt
MIDI Pitch → Hz	76.midicps // (returns ~659.25 Hz)	Converts MIDI note 76 (E5) into frequency
Percussive Env	EnvGen.kr(Env.perc(0.01, release), ...)	Shapes volume decay without sustaining
Stereo Spacialization	Pan2.ar(signal, pan)	Routes mono signal to 2-channel stereo bus
Harmonic Layering	SinOsc.ar(freq) + SinOsc.ar(freq * 2 * 0.2)	Blends fundamental with 2nd harmonic
Sequential Clock	0.25.wait // inside Routine({ ... })	Controls tempo and rhythm between notes

> **Offline Debugging Checklist:**
>
> • Did scsynth server boot cleanly? (s.boot;)
> • Are all synths self-releasing, or do you still see node counts rising on the server meter?
> • Is the Routine wrapped with .play at the end of the evaluation block?
> • Do the frequencies sound like the iconic *Für Elise* motif?